

SD TCL - Printing and Spooling

Print units at the command prompt, keeping a job open, and capturing a session instead of printing it.

Printing is set up at the prompt and used from a program. **This page is the prompt half:** the verbs that point a print unit somewhere, hold it open, and capture what a session displays. The statements a program uses — `print on`, `heading`, `printer file` — are in SD Basic - Printing, and nothing here repeats them.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** something you type as it stands.

Print units, and where they point

Unit 0 is the terminal. Every other unit is somewhere you have sent it. `setptr` is how you look and how you set:

```
:setptr display
Unit Width Depth Tmgn Bmgn Mode Options
  0    80    66   0    0    1
```

COLUMN

Unit	the print unit number a program names in <code>print on n</code>
Width, Depth	the page, in characters and lines
Tmgn, Bmgn	top and bottom margins, in lines
Mode	what the unit does with the output
Options	everything else that has been set on the unit

The depth is 66 and the terminal is not 66 lines. Unit 0's page is a printed page, not your screen — it is the size a report is paginated to when it goes to a printer. The screen's own size is `term`, which is a different setting on a different page: SD TCL - The Terminal and the Session.

Setting a unit

```
setptr unit, width, depth, top.margin, bottom.margin, mode {, options}
setptr default, width, depth, top.margin, bottom.margin, mode {, options}
setptr display
setptr unit, display
```

The positional arguments are all optional — `setptr 1,,,,3` sets only the mode of unit 1 — and `nodefault` keeps the values you did not name rather than resetting them.

`setptr` **asks before it changes a unit, and** `brief` **is how you stop it.** That confirmation is a prompt, so a `setptr` inside a phantom, a script or an API session **hangs waiting for an answer nobody is there to give.** Give it `brief` in anything that is not a person at a keyboard.

The options are many and most are printer-specific. The ones worth knowing at the prompt:

<code>as</code> <i>name</i> , <code>as pathname</code> <i>path</i>	the name or path a mode 3 unit writes to
<code>at</code> <i>printer</i>	the Windows printer to send to
<code>brief</code>	suppress the confirmation prompt
<code>keep.open</code>	hold the unit open between jobs — see below
<code>copies</code> <i>n</i> , <code>landscape</code> , <code>portrait</code> , <code>duplex</code>	passed to the Windows spooler
<code>newline</code> <code>cr</code> <code>lf</code> <code>crlf</code>	the line ending written
<code>nfmt</code>	send the data with no formatting at all
<code>style</code> <i>name</i>	apply a report style

`paper.size`, `cpi`, `pcl`, `symbol.set`, `weight` and `overlay` are the printer-formatting set and behave as they always did.

The same settings in keywords: `printer`

```
printer {unit} query          report one unit
printer {unit} at printer.name send it to a Windows printer
```

```
printer {unit} file file.name record    send it to a record
printer {unit} width n | lines n        the page
printer {unit} top.margin n | bottom.margin n | left.margin n
printer {unit} keep.open | close
printer {unit} reset                    back to the defaults
```

`printer` **does the same job as** `setptr` **and does not ask first.** That is the reason to know it exists: `setptr` confirms before it changes a unit unless you remember `brief`, and `printer` has no confirmation at all. **It is the form to use from a phantom, a script or an API session.**

More than one keyword may be given, in any order. The unit number comes before the keywords and **defaults to 0**, the terminal.

`query` **is the per-unit report**, and it names things `setptr display`'s columns do not:

```
:printer query
PRINT UNIT 0
  Page width      : 80
  Lines per page  : 66
  Top margin      : 0
  Bottom margin   : 0
  Left margin     : 0
  Mode            : 1
  Printer         : (Default)
```

```
:printer 1 query
PRINT UNIT 1
  Page width      : 80
  Lines per page  : 66
  Top margin      : 0
  Bottom margin   : 0
  Left margin     : 0
  Mode            : 3
  Target          : $hold P1
```

The last line names the destination, and its wording tells you whether one has been set:

<code>Printer : (Default)</code>	mode 1, no printer named — the Windows default
<code>Printer : <i>name</i></code>	mode 1, that Windows printer
<code>Target : \$hold P <i>n</i></code>	mode 3, nothing named — the unit's own hold record

Pathname : `path`

mode 3, that destination

`setptr display` shows none of this; it is a table of numbers.

Setting and putting back in one session — the two `query` reports are cut to the lines that moved:

```
:printer 1 width 132 lines 60
:printer 1 query
  Page width      : 132
  Lines per page  : 60
:printer 1 reset
:printer 1 query
  Page width      : 80
  Lines per page  : 66
```

Nothing is printed on the way through — the settings are the report, and `query` is how you see them. `reset` puts width, depth and all three margins back to their defaults **and clears the destination as well**: the printer name on unit 0, the file and record on any other unit. (*Unit 1 above had no destination set either side of the reset, so its last line reads `Target` both times — that is the wording for "nothing named", not evidence that `reset` left it alone.*)

Invalid print unit number

`printer` with nothing after it, or a unit outside the range

Unexpected token (%1)

a keyword it does not know

Printer name not recognised

`at` with a name Windows does not have

Holding a unit open

```
sp.open
sp.close
```

By default a unit finishes its job and releases when the program closes it, so two reports become two print jobs. `sp.open` sets the keep-open flag so they become one; `sp.close` clears it again. The flag shows up in `setptr`'s Options column, which is the way to check it took:

```
:setptr display
  0   80   66   0   0   1
:sp.open
:setptr display
```

```
0    80    66    0    0    1 KEEP.OPEN
:sp.close
:setptr display
0    80    66    0    0    1
```

Neither verb prints anything. The Options column is the only report you get, and it is worth looking at — a keep-open flag left set is why a report occasionally arrives stapled to the one before it.

Printing records

```
spool file.name record.id... {lines n m} {lnum} {lptr n}
```

With no ids it prints select list 0, so the usual shape is a `select` followed by a `spool`. With no file name at all it refuses rather than guessing:

```
:spool
File name required
```

<code>lines</code> <i>n m</i>	print only lines <i>n</i> to <i>m</i> of each record
<code>lnum</code>	number the lines
<code>lptr</code> <i>n</i>	send to print unit <i>n</i> rather than the default

Looking at what is queued

`sp.view` shows the records in `$hold` and offers to print them.

It is a full-screen form and cannot be driven down a pipe. It asks for a printer number, then whether to print, then a confirmation — three prompts, so a script that runs it stops and waits. Like the screen editors, it wants a person at a terminal. Everything on this page except `sp.view` can be scripted.

`$hold` is an ordinary directory file in the account, so a script that needs to work with held reports can read it with `select`, `list` and `spool` instead.

Capturing a session instead of printing it

```
como on record.name
como off
```

como writes everything the session displays into a record in the `$como` file, which it creates in the account the first time you use it. End to end:

```
:como on zzcomo
COMO file activated to zzcomo
:who
11 DON
:date
Thursday, 27 August 2026 01:13pm
:como off
COMO file deactivated
```

and the record afterwards:

```
:ct $como zzcomo
$como zzcomo
1: COMO file activated to zzcomo
2:
3: 11 DON
4:
5: Thursday, 27 August 2026 01:13pm
6:
```

It captures the prompts and the echoes, not just the output. The record is a transcript of the session, which is what makes it useful for showing somebody what you did — and it means a COMO of a long run is larger than the output alone. **como off** is captured too, as the last thing before the file closes.

To capture one command rather than a whole session, `execute ... capturing` in a program is the better tool — see SD Basic - Program Control.

Report styles

```
report.style
report.style name
```

```
report.style off
```

A report style is a stored set of print settings applied by name. With no argument the verb says what the default is:

```
:report.style  
No default report style has been set
```

which is what a fresh account reports. `off` clears it again.

Printing on this port

A named printer is a Windows printer, and printing goes to the Windows spooler rather than to a POSIX print command. There is no `lp` and no PostScript pipeline. The consequences for a report that assumed either are set out in SD Basic - Printing, and they apply the same way to a unit set up with `setptr`.

See also

[SD Basic - Printing](#) · [SD TCL - The Terminal and the Session](#) · [SD TCL - The Query Processor](#).
